

Simplified Implementation of Individual Confidential Computing of Polynomials over Non-Uniform Information [[1](#)]

Advised by Netanel Raviv

Daniel Palmer

Spring 2026 Independent Study Final Report

May 6, 2026

Contents

1	Introduction	1
2	Background	1
2.1	Motivation for the ICC Scheme	1
2.2	Perfect Subset Privacy	3
2.3	Polynomial Computation with Perfect Subset Privacy	3
2.4	ICC: Extension to Non-Uniform Data	5
3	2024 (Uniform) Implementation	6
3.1	System Architecture	6
3.2	Client Design	7
3.3	Server Design	8
3.4	Worker Design	9
3.5	Testing	9
4	Adapting to Full ICC	10
4.1	Shifting from Reed Solomon to Random Linear Code	10
4.2	Entropy Calculation and Derivation of Key Size	10
4.3	Information Set Enumeration at Scale	12
4.4	Testing	12
5	Further Work	17
5.1	Straggler Resilience via Information Super-Sets	17
5.2	Case Study	18
	References	19

1 Introduction

I am an undergraduate BS/MS student studying Computer Science who wants to get research experience before figuring out my post-graduation path. After taking Professor Raviv's Introduction to Machine Learning and then Coding and Information Theory classes, I strongly resonated with his work and wanted to pursue research with him. Through discussion of my interests and the time that I have for my project, he suggested that I create a concrete implementation of the scheme described in his recent paper Individual Confidential Computing of Polynomials over Non-Uniform Information [1]. This would bridge my experience/interest in machine learning with information theory that fit what I wanted out of a research project. I started this project in January 2026 with the plan that I would develop it over the next year and a half into my master's thesis.

We discussed how I would divide this work so that I could have a reasonable goal for what I would have finished this semester for my Independent Study. We concluded that my work this semester should produce a concrete, simplified toy example of the scheme that proves it works, which I will then build out over the next year to be fully robust. Specifically, the key simplification that I am making is I am not considering straggler tolerance. Further, I am using small, randomly generated data with small parameters such that I can run and test this scheme solely on my personal computer. I look forward to further developing this project over the next year.

2 Background

2.1 Motivation for the ICC Scheme

The proliferation of artificial intelligence and machine learning has fundamentally transformed how data is handled. As model complexity and dataset scale continue to grow, the computational demands have exceeded the infrastructure available to all but the largest institutions. This has driven heavy reliance upon cloud computing services such as Amazon Web Services, Microsoft Azure, and Google Cloud, for example, which offer on-demand access to powerful computation power at a fraction of the cost of building equivalent infrastructure. For many smaller companies and research institutions, such

services are not just convenient but required to remain competitive.

Unfortunately, this reliance on external computation introduces tension regarding data privacy. Using cloud computing generally requires uploading the raw dataset to the service provider, exposing the data to an entity outside of the user’s control. For organizations constrained by regulations such as HIPAA, or those whose competition depends on their data being confidential, this exposure may be unacceptable. The result is that privacy sensitive organizations are systematically excluded from the computational resources that would allow them to take advantage of modern AI efficiency. Thus, there is need for secure ways to use cloud computing to democratize the space.

The natural response is to encrypt the data before uploading it, and significant research has gone towards analyzing just this. The standard approach, homomorphic encryption (HE), instead incurs massive overhead and does not allow common learning algorithms to be performed directly on the encrypted data [2]. Further, HE uses computational rather than information-theoretic privacy guarantees. The scheme that I am implementing takes an alternative approach grounded in information-theoretic security. The goal is to encode data using a short random key such that the encoded version provably leaks no information about any small subset of the original data, regardless of the computational power of the adversary. This guarantee is formalized through the notion of perfect subset privacy (PSP), which ensures zero mutual information between the encoded data and any r -element subset of the original where r is a tunable security parameter. This notion of PSP kickstarted the line of three papers, each building off of the last, which result in my implementation. Information-theoretic guarantees of the kind that PSP gives are strictly stronger than the computational ones such as HE. They hold unconditionally, without any assumption about the adversary’s resources.

Within this information-theoretic framework, prior work on privacy in distributed computation has focused on protecting data held by the system administrator from adversarial worker nodes inside the service provider. Crucially, this assumes that the system administrator is fully trusted. This is problematic when the user wants privacy against the entire service provider.

The scheme implemented in this project, introduced in [3] and extended to non-uniform data in [1], addresses this gap by shifting the privacy barrier from the boundary between the administrator and its workers to the boundary between the user and the

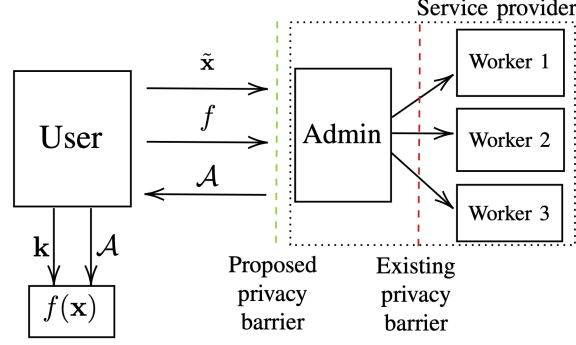


Figure 1: Updated Scheme Privacy Barrier vs Existing Privacy Barrier [3]

service provider as a whole. This shift is illustrated in Figure 1.

2.2 Perfect Subset Privacy

The foundational privacy notion this project relies on is PSP as previously mentioned. It is defined as follows: consider a user holding data $x = (x_1, \dots, x_n) \in \mathbb{F}_q^n$. x is encoded into $\tilde{x}$ by a privatization scheme. Rather than requiring that $\tilde{x}$ reveals nothing about x in its entirety (which would be useless because the privatized data could not be used for inference on the real data), PSP instead requires that $\tilde{x}$ reveals nothing about any r -subset of x .

For a fixed $r \in [n]$, an encoding procedure satisfies r -subset privacy if:

$$I(X_R; \tilde{X}) = 0 \quad \forall R \in \binom{[n]}{r}$$

The parameter r is tunable: larger r gives stronger privacy protection at the cost of a larger key [2].

2.3 Polynomial Computation with Perfect Subset Privacy

Deng, Ramukar, and Raviv [3] use PSP to develop an encoding scheme achieving PSP. This scheme is straightforward. Let $\tilde{x} \triangleq x + kG$ where $k \sim \text{Unif}(\mathbb{F}_q^m)$ and $G \in \mathbb{F}_q^{m \times n}$ is the generator matrix of a linear code C whose dual code $C^\perp$ satisfies $d_{\min}(C^\perp) \geq r + 1$ where $d_{\min}$ is the minimum hamming distance. This condition ensures that every r columns of G are linearly independent which makes the privacy proof work.

The key size m is bounded by r via the Singleton bound. This bound is tight as

equality can be achieved by taking C to be a Maximum Distance Separable (MDS) code which achieves $m = r$. In particular, Reed-Solomon codes with Vandermonde generator matrices achieve this optimum and this is the generator matrix that I used when implementing the uniform scheme.

After data storage, the user wants to calculate a polynomial $f(x)$ which is a multivariate polynomial $f : \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ of total degree at most d and individual degree at most $q - 1$. The key to keeping the computation identical while preserving privacy is defining a new polynomial $g : \mathbb{F}_q^n \rightarrow \mathbb{F}_q$ as $g(t) \triangleq f(\tilde{x} - tG) \forall t \in \mathbb{F}_q^m$. Crucially, this gives $g(k) = f(\tilde{x} - kG) = f(x + kG - kG) = f(x)$. So, recovering $f(x)$ is nothing beyond evaluating the polynomial g at the key k . This means that the service provider gets the evaluations of g at a set of points and returns these to the user who interpolates to get g and evaluate $g(k)$.

Considering what evaluation points to take is incredibly interesting. The paper uses Reed-Muller codes to optimize beyond the trivial answer of giving all q^m evaluations. It uses information sets which are sets of points such that “any codeword can be reconstructed in full by observing its entries on the information set” [3]. An explicit information set for $\text{RM}_q(d, m)$ is given in Definition 3 as $I_{d,m} \triangleq \{(\alpha_{i_1}, \dots, \alpha_{i_m}) \mid \sum_{k=1}^m i_k \leq d, 0 \leq i_k \leq q - 1\}$ [3]. Because g is a degree- d polynomial in m variables, knowing g ’s evaluations at an information set is enough to interpolate all of g .

If I were to be incorporating straggler tolerance into my scheme then this simple information set would be insufficient because it requires all evaluation points to be returned. The paper combats stragglers through a new tool: an information super-set which is a multiset that is resilient against S stragglers because any subset of the multiset without S elements is a complete information set. However, because I am making the simplification that I am not considering stragglers currently, the simple information set given in Definition 3 is sufficient.

In one paragraph, the scheme proceeds as follows. In the storage phase the user uploads $\tilde{x} = x + kG$ and the server distributes $\{\tilde{x} - tG \mid t \in I_{d,m}\}$ to worker nodes. In the computation phase, the user sends f to the administrator, workers evaluate f on their data and return results, and the administrator sends $A = \{f(\tilde{x} - tG)\}_{t \in I_{d,m}}$. The user solves a linear system in order to interpolate g , evaluates it at k , and obtains $f(x)$.

The full ICC scheme [1] builds directly off of this setting so I decided it would be the

best place to begin my implementation. I started by building a complete toy implementation (again not considering stragglers) of this scheme described in [3]. I then built off of it to realize the full ICC scheme.

2.4 ICC: Extension to Non-Uniform Data

The scheme of [3] assumes that the user's data x is uniformly distributed over $\mathbb{F}_q^n$. For real datasets this almost never holds and the privacy proof breaks down without it. Tarnopolsky et al. [1] identify that this is true because achieving zero leakage for non-uniform x requires a key whose entropy matches that of the data which would nullify the storage benefits of keeping a key instead of x . They instead target negligible leakage where $I(X_R; \tilde{X}) \leq \varepsilon_c$ for negligible ε_c .

The central tool is distribution smoothing. Intuitively, if the encoding $\tilde{x} = x + kG$ can produce an output that is nearly uniformly distributed regardless of the distribution of x , then the privacy argument from the uniform case carries through approximately. The key tool is p -entropy which is defined for a random variable $X \in \mathbb{F}_q^m$ and $p \geq 2$ as $H_p(X) = \frac{1}{1-p} \log_q(\sum_{x \in \mathbb{F}_q^m} p^p(x))$. If this quantity is high then X carries much uncertainty (e.g. more uniform) while if this quantity is low then X is less uniform.

The key result (Theorem 1) is that a randomly chosen generator matrix $G \in \mathbb{F}_q^{m \times n}$ (as opposed to a structured generator matrix of an MDS code as in [3]) performs smoothing provided m is large enough relative to the system parameters and p -entropy of the data. This gives the following theorem for the size of the key k , $m \geq n + p + \log_q(\frac{1}{\varepsilon}) - H_p(X) + \max_R H_p(X_R)$. Taking such an m gives, with probability at least $1 - \frac{1}{a}$, the encoding $\tilde{X} = X + kG$ satisfies $I(\tilde{X}; X_r) \leq \varepsilon_c \forall R \in \binom{[n]}{r}$ where ε_c is a closed-form quantity depending on a, p, q, ε , and $\max_R H_p(X_R)$.

The storage benefits of this can be seen by taking $H_p(X)$ as large as possible, which

is realized when X is uniform. Uniform X gives $p_X(x) = \frac{1}{q^n}$. So, this gives:

$$\begin{aligned}
 H_p(X) &= \frac{1}{1-p} \log_q \left(\sum_{x \in \mathbb{F}_q^n} p^p(x) \right) \\
 &= \frac{1}{1-p} \log_q \left(\sum_{x \in \mathbb{F}_q^n} \left(\frac{1}{q^n} \right)^p \right) \\
 &= \frac{1}{1-p} \log_q (q^n q^{-np}) \\
 &= \frac{1}{1-p} \log_q (q^{n(1-p)}) \\
 &= \frac{1}{1-p} n(1-p) \\
 &= n
 \end{aligned}$$

The same argument applies for $H_p(X_R) = r$ when X_R is uniform. So, the final value for m is found in $m \geq n + p + \log_q(\frac{1}{\varepsilon}) - n + r = p + r + \log_q(\frac{1}{\varepsilon})$. Thus, if X is uniform, m depends solely on instance parameters and not input size, which means the input size can be arbitrarily large without key size blowing up. So, this encoding scheme overcomes the storage problems when accepting negligible mutual information instead of zero. This comes at a cost, however, because this optimal construction only exists when X is uniform and less-uniform distributions push m larger.

The computation and decoding pipeline—the $g(t) \triangleq f(\tilde{x} - tG)$ construction, RM information set evaluation, and linear system solve—is inherited from [3] unchanged. The only modifications relative to the uniform scheme are the method of generating G , the procedure for computing the required m , and the utilities to track information leakage.

3 2024 (Uniform) Implementation

3.1 System Architecture

The uniform scheme implementation models the three-party distributed system described in [3] as three Python classes: **Client**, **Server**, and **Worker**. A fourth class, **SystemContext**, acts as a parameter container passed to every component ensuring that the same parameters are used consistently across all parties. There is also a separate

utilities file, `utils.py`, which contains general utilities for the scheme.

The `SystemContext` contain 6 elements, q, n, m, d , and r . q is the field size, n is the data length, m is the key length, d is the maximum polynomial degree being computed over the data, and r is the privacy parameter.

One early obstacle I had to manage was computations over finite fields instead of real numbers. Standard Python arithmetic cannot be used. I found the galois library [4] which is a NumPy extension for Galois (finite) fields to be a perfect tool for this.

3.2 Client Design

The `Client` has three methods: `encode_data`, `_evaluate_monomials`, and `decode_result`. It is responsible for encoding during the storage phase and decoding during the computation phase. It keeps only the secret key $\mathbf{k}$ after encoding which reflects the storage model of [3].

The `Client` constructor takes `SystemContext` and `Galois` field objects which are stored as attributes.

The `encode_data` method implements Equation (1) of [3] directly. It takes as parameters $\mathbf{x}$ which is the data and $\mathbf{G}$ which is the generator matrix being used to mask $\mathbf{x}$. $\mathbf{G}$ is a Vandermonde matrix which is constructed by the utility function `generate_vandermonde_G` in `utils.py`. This function selects takes a `Galois` field (GF), m as the number of rows which is the key length, and n as the number of columns which is the data length as parameters. It then selects n distinct, non-zero elements $\alpha_1, \dots, \alpha_n \in \mathbb{F}_q$ and sets $G_{i,j} = \alpha_j^{i+1}$ for $i \in 1, \dots, m$ and $j \in 1, \dots, n$. This produces a Reed-Solomon generator matrix which obtains the optimal $m = r$ bound (the minimum possible m constrained by the Singleton bound). Back to `encode_data`, it first generates $\mathbf{k}$ with `self.GF.Random(self.context.m)` which it stores as an attribute. It then calculates $\tilde{x} = x + kG$ directly using `numpy.matmul` and returns $\tilde{x}$.

The `decode_result` method implements the interpolation step of the scheme and is the most mathematically involved aspect of the client. It takes as parameters the server results $A = \{f(\tilde{x} - tG)\}_{t \in I_{d,m}}$ and the points generating these results $\{t | t \in I_{d,m}\}$. We know that $g(t) = f(\tilde{x} - tG)$ is a multivariate polynomial of total degree at most d in m variables. Because the client receives evaluations of g at every point in $I_{d,m}$, it follows from Lemma 2 of [3] that the coefficients of g are uniquely deter-

mined in the monomial basis. The **Client** first gets the exponent tuples which are obtained by calling `get_information_set` in `utils.py`. This function builds the information set $I_{d,m}$ given by Definition (3) in [3] by iterating over every combination in `itertools.product(range(context.q), repeat=m)` and returning each combination that satisfies the sum of the combination being at most `context.d`. Let λ be the size of this information set (which follows from the $\lambda(q, d, m)$ notation in [3]). For each point t that `decode_result` is given, it builds the evaluation matrix $M \in \mathbb{F}_q^{\lambda \times \lambda}$ where each row holds monomial evaluations for t . These evaluations are computed by `_evaluate_monomials` which iterates over each exponent tuple e and accumulates $\prod_i t_i^{e_i}$ over the field. **Client** then solves the system $Mc = A$ using `numpy.linalg.solve` to get the coefficient vector c of g . Finally, **Client** recovers $f(x) = g(k)$ by evaluating k at the exponents with the same `_evaluate_monomials` and then multiplying by the coefficients c and summing to get the final decoded answer.

It is worth noting that M is guaranteed to be invertible because $I_{d,m}$ is an information set which is the property that makes the linear system uniquely solvable.

3.3 Server Design

The **Server** class models the system administrator described in [3]. It has three methods: `store_data`, `_get_evaluation_points`, and `compute_request`. It is responsible for receiving and distributing encoded data during the storage phase and coordinating polynomial evaluation over the workers during the computation phase.

The **Server** constructor takes **SystemContext** and **Galois field (GF)** objects which are stored as attributes.

The `store_data` method takes both the generator matrix G and encoded data $\tilde{x}$ as parameters. It stores G as an attribute and then distributes $\tilde{x}$ to workers. The workers each receive an evaluation point which are generated by `_get_evaluation_points` which wraps the result from `get_information_set` in **GF** because the server requires finite field arithmetic. These points are stored as an attribute to give back to the **Client** later. For each evaluation point t (member of the information set $I_{d,m}$, the server distributes $\tilde{x} - tG$ where tG is obtained by multiplying t and G with `numpy.matmul` to a **Worker** and stores the worker in a **Worker** list attribute. This is consistent with step (ii) of the scheme in [3]. The server never has access to x or k directly, only $\tilde{x}$ and G which is consistent with the

privacy model.

The `compute_request` method is used in the computation phase and takes a polynomial f . Its core functionality is querying all of the workers for their evaluations of f . It aggregates these results and returns a tuple where the first element contains a list of evaluation points given to each `Worker` and the second element contains a list of the results. The client needs both evaluation points and results in order to build the decoding matrix M .

3.4 Worker Design

The `Worker` class is intentionally very minimal. Each worker holds a single piece of shifted data $\tilde{x} - tG$ for $t \in I_{d,m}$ and has a single method `evaluate`.

The `Worker` constructor takes the piece of shifted data and stores it as an attribute.

The `evaluate` method takes a polynomial f and returns its computation over the `Worker`'s shifted data.

3.5 Testing

The main file for the project contains a script that runs an instance of the scheme with random data. This script performs setup by initializing a `SystemContext` instance with the parameters, creating a Galois field `GF`, initializing a `Client` and `Server` with these parameters and field, generating random data $\mathbf{x}$, and generating a Vandermonde matrix $\mathbf{G}$ to use for encoding. It then sends $\mathbf{x}$ and $\mathbf{G}$ to the client to encode, receiving the encoded $\tilde{x}$. It then defines the polynomial it wants calculated and sends this polynomial to the server to compute. It receives the points $t \in I_{d,m}$ that $f(\tilde{x} - tG)$ was evaluated at along with their evaluations. It then sends these points and results back to the client to decode which returns the decoded answer. Finally, this script computes the polynomial over the actual data and compares to see if the scheme was successful.

I ran this script with varying polynomials and parameter combinations and they were all successful. I am not going to list all of this out here because, due to the fact that the full ICC implementation inherits these classes and only changes $\mathbf{G}$ and the parameter $\mathbf{m}$, I include all of these tests in my exhaustive testing below.

4 Adapting to Full ICC

4.1 Shifting from Reed Solomon to Random Linear Code

There are two key changes that have to be made when considering non-uniform data in the shift from the scheme in [3] to the scheme in [1]. The first of which is replacing the construction of G from a Vandermonde matrix, which was chosen because it is the generator matrix of an MDS code (Reed-Solomon), with a generator matrix chosen uniformly at random. This shift in code is straightforward, I created another utility function `generate_random_G` which takes the same parameters as `generate_vandermonde_G` and returns a randomly generated $m \times n$ array using `GF.Random`. The only other change is replacing the call to `generate_vandermonde_G` with `generate_random_G`.

4.2 Entropy Calculation and Derivation of Key Size

The second key change is removing m as a free parameter of the system and instead deriving it from other parameters and the entropy of the data. The first amendment is changing `SystemContext`. In terms of free parameters, I removed m and added p which is the value for p -entropy, `epsilon` which is the smoothing “budget” and controls how strict the smoothing must be, and a which is the confidence parameter of successful smoothing. I also added the following three parameters all set to `None` because they must be calculated over the data: $H_p X$ which is the p -entropy of the data, $\max H_p X_R$ which is the max p -entropy of any r -subset of the data, and m which is the key length.

Next, I added utility functions for entropy calculation and privacy guarantees. The first such function is `compute_p_entropy`. This function takes data x , field size q , and entropy order p and returns the p -entropy of the data. The first step is to construct a probability distribution for the data by iterating over each element, counting how many times it occurs, and creating a probability for it by dividing occurrence rate by total. It takes this distribution and then simply calculates the p -entropy using Python math functions according to Appendix A in [1]

The second utility function I added is `compute_max_subset_p_entropy`. This function takes data x , field size q , entropy order p , and subset size r as parameters and returns the maximum p -entropy contained within any r -subset of the data. It does this by iter-

ating over every r -subset obtained by calling `itertools.combinations(range(n), r)`, taking the subset of x corresponding to these indices, computing the p -entropy by calling `compute_p_entropy`, and tracking the maximum.

After the two entropy functions are created, all of the tools exist to compute m . Thus, the next utility function I added was `compute_required_m` which takes `SystemContext` as a parameter and returns the minimum possible value of m according to the Theorem 1 bound [1]. At this point, both the entropy of the data and max entropy of any r -subset have been calculated and stored in the context. So, the first thing the function does is ensure that these values exist. Given they do, the function then calculates m directly from the

$$m \geq n + p + \log_q\left(\frac{1}{\epsilon}\right) - H_p(X) + \max_R H_p(X_R)$$

formula in Theorem 1. It first calculates `log_q_inv_eps=log_q(1/epsilon)` with `math.log` and then calculates the quantity `n + p + log_q_inv_eps - H_p_X + max_H_p_X_R` where n and p are also in context. Finally it returns the max between r from context and the ceiling of this calculated quantity because m must also abide by the Singleton bound so is at least r and must be an integer.

Because the scheme in [1] now has nonzero information leakage, it is necessary to track how much information is being leaked. The final utility function I added does just that. This function is `compute_leakage_bound` and it computes the information leakage bound ϵ_c according to

$$\epsilon_c = \frac{p}{p-1} \log_q\left(1 + a \cdot 2^{\frac{2p-1}{p}} (1 + q^{-\max_r H_p(X_R)}) \epsilon^{\frac{1}{p}}\right)$$

given in Theorem 1 in [1]. It takes `SystemContext` as a parameter and returns the computed ϵ_c . This equation requires `max_H_p_X_R` to be set in the context which is the first check that it makes. It then computes ϵ_c directly using Python math from p, q, eps and a from the context.

All of these functions are used in main and testing scripts to produce a successful encoding and to verify that the encoded data has more entropy than the original non-uniform data. After I finished these functions, my code now fully implements the ICC scheme described in [1] under my no stragglers simplification.

4.3 Information Set Enumeration at Scale

I began testing the uniform scheme with $q = 31$ and $n = 10$ to have a wide variety of inputs. However, when I began testing the ICC implementation with these parameters and skewed data the computation time exploded. I looked into why and realized that my `get_information_set` function using `itertools.product` iterated over every possible combination which required calculating $O(q^m) = O(31^{17})$ which is in the order of 10 septillion combinations. This was tolerable before because I had $m = r$, which gave $O(q^m) = O(31^r)$ which is reasonable-enough to compute for small r as I had. So, in order to combat this, I revised my `get_information_set` implementation to recursively check only combinations that may possibly sum to at most d . I did this by tracking the “budget” remaining for a combination being created and never exceeding this budget. This means that whole subtrees which are useless to check get pruned. This runs in $O(|I_{d,m}|) = O(\lambda(q, d, m)) = O(\binom{m+d}{d})$ time. For the same parameters, I was testing with $d = 2$ and calculated $m = 17$. So, it requires calculating 171 combinations instead of 10 septillion. This is a massive improvement as it does not waste computation on combinations that would never be used.

4.4 Testing

In order to verify my scheme I changed the main script from the previous, uniform scheme. I amended it so that the data distribution is skewed where the first a values share 90% of the mass and the remaining $n - a$ values share the remaining 10%. I also added entropy and leakage bound calculations so that I can test how well the scheme is performing. Throughout the development of my implementation I performed simple tests with the main script to ensure that the implementation is not failing. I tested over a variety of parameter combinations and, once I was happy with my result, decided to make a fully comprehensive test suite. I used both Claude and Gemini to generate portions of the test suite according to the different situations I wanted to test and validated their output. This suite is in the file `test_icc.py` and it is split up into six sections that each test a different aspect of the scheme.

Section 1: Correctness

Table 1 confirms that the Reed-Muller information-set decoding pipeline produces exact results across all tested polynomial forms. All 10 trials pass for every polynomial, validating correctness for degrees 1 through 4. An interesting note is that the computation times exploded as the polynomial degree increased. This is expected as calculating the information set becomes more involved. Unfortunately, it means that any polynomial higher than degree 4 does not work to test on my personal computer.

Table 1: Polynomial variety: 10 independent trials per polynomial ($q = 31$, $n = 10$ or 6 , $\varepsilon = 10^{-6}$, skew = 0.9).

Polynomial	Trials	Pass	Avg. time (ms)
$f = x_0^2 + x_1x_2$	10	10	1380.7
$f = x_0 + x_1 + x_2$	10	10	1363.2
$f = x_0x_1 + x_2x_3$	10	10	1389.8
$f = x_0^2 + x_1^2 + x_2^2$	10	10	1411.7
$f = x_0x_1x_2$	10	10	19842.5
$f = x_0x_1x_2x_3$	10	10	448252.4

Table 2 shows correctness across varying system parameters (q, n, d, r) , and Table 3 confirms stability over 50 independent random key draws.

Table 2: Parameter variety: $f = x_0 + x_1$, 5 trials per configuration.

(q, n, d, r)	Trials	Pass	Avg. time (ms)
(31, 8, 2, 4)	5	5	887.5
(37, 10, 2, 3)	5	5	1167.9
(41, 6, 3, 2)	5	5	13028.0
(31, 12, 1, 5)	5	5	12.9

Table 3: Random key stability: $f = x_0^2 + x_1x_2$, $q = 31$, $n = 8$, $d = 2$, $r = 4$, 50 independent key draws.

Trials	Pass	Avg. time (ms)
50	50	902.2

These correctness tests provide comprehensive testing that the uniform scheme functions as intended.

Section 2: Theorem 1 Bound Validation

Theorem 1 requires

$$m \geq n + p + \log_q\left(\frac{1}{\varepsilon}\right) - H_p(X) + \max_R H_p(X_R)$$

Table 4 verifies that `compute_required_m()` returns exactly $\lceil \text{RHS} \rceil$ (marked **T** for tight) across all tested parameter combinations, confirming that the implementation satisfies the bound with equality.

Table 4: Theorem 1 bound verification across parameter combinations. **T** denotes a tight bound ($m = \lceil \text{RHS} \rceil$).

$(q, n, r, p, \varepsilon)$	$H_p(X)$	$\max_R H_p(X_R)$	m	RHS
(31, 8, 3, 2, 10^{-4})	0.241	0.320	13 T	13
(31, 10, 5, 2, 10^{-6})	0.253	0.371	17 T	17
(31, 12, 4, 2, 10^{-6})	0.345	0.404	19 T	19
(31, 8, 3, 3, 10^{-5})	0.124	0.320	15 T	15
(37, 10, 4, 2, 10^{-6})	0.215	0.272	16 T	16

Section 3: Privacy and Leakage Analysis

Table 5 shows that ε_c decreases strictly as ε decreases, consistent with Theorem 1: tighter smoothing yields smaller information leakage.

Table 5: Leakage bound ε_c as a function of ε ($q = 31$, $n = 10$, $r = 5$, $p = 2$, skew = 0.9, $H_p(X) = 0.332$, $\max_R H_p(X_R) = 0.469$).

ε	m	ε_c
10^{-2}	14	0.57784776
10^{-4}	15	0.09129453
10^{-6}	17	0.00980095
10^{-8}	18	0.00098755
10^{-10}	19	0.00009883

Table 6 sweeps skewness from 0.3 to 0.95 at fixed $\varepsilon = 10^{-6}$. The leakage bound ε_c is non-decreasing in skewness, confirming that more non-uniform data requires a higher privacy cost, as expected from the entropy term $\max_R H_p(X_R)$ in Theorem 1.

Table 7 sweeps the confidence parameter a , which controls the probability guarantee $\Pr[\text{ICC holds}] \geq 1 - 1/a$. The leakage bound ε_c is strictly increasing in a , reflecting the

Table 6: Leakage bound ε_c as a function of data skewness ($q = 31$, $n = 10$, $r = 5$, $p = 2$, $\varepsilon = 10^{-6}$).

Skew	$H_p(X)$	$\max_R H_p(X_R)$	m	ε_c
0.30	0.499	0.469	16	0.00980095
0.50	0.416	0.469	17	0.00980095
0.70	0.416	0.469	17	0.00980095
0.80	0.351	0.371	17	0.01044852
0.90	0.190	0.190	17	0.01238692
0.95	0.226	0.298	17	0.01109537

trade-off: higher confidence in the probabilistic smoothing guarantee comes at the cost of a looser worst-case leakage bound.

Table 7: Leakage bound ε_c and probability guarantee as functions of a ($q = 31$, $n = 10$, $r = 5$, $p = 2$, $\varepsilon = 10^{-6}$, skew = 0.9, $m = 17$ throughout).

a	Prob. guarantee	ε_c
2	≥ 0.500	0.00394019
5	≥ 0.800	0.00980095
10	≥ 0.900	0.01943970
20	≥ 0.950	0.03825144
50	≥ 0.980	0.09129453

Section 4: Entropy Sensitivity

Table 8 confirms that $H_p(X)$ is decreasing in p for fixed data, consistent with the property of p -entropy stated in Appendix A.

Table 8: $H_p(X)$ as a function of p for fixed data ($q = 31$, $n = 8$, skew = 0.9).

p	$H_p(X)$
2	0.310962
3	0.307396
4	0.304406
5	0.301925
6	0.299877

Table 9 shows that after linear-code encoding, $H_p(\tilde{X}) > H_p(X)$ in all cases, confirming that the encoding moves the distribution toward uniform as required by Definition 2.

Table 9: Entropy before and after encoding ($q = 31$, $n = 8$, $r = 4$, $p = 2$).

Skew	$H_p(X)$	$H_p(\tilde{X})$
0.70	0.2623	0.4875
0.90	0.2206	0.5406
0.95	0.1518	0.5406

Section 5: Edge Cases

With $r = 1$ (minimal privacy parameter), the scheme produces $m = 14$ and decodes correctly. With near-deterministic data (99% mass on a single symbol), the scheme correctly inflates m to 15 to compensate for the extremely low entropy, and still decodes correctly.

Section 6: Massive Datasets ($m \ll n$)

Over the course of my testing I noticed that, given I was testing with small n so that `compute_p_entropy` would run reasonably quickly on my machine, m was always greater than n . As shown in my analysis in Section 2.4, I knew that the scheme allows m much smaller than n and I wanted to verify this. So, I did more research into p -entropy and found that it is the same as Rényi entropy. I looked into Rényi’s 1961 work On Measures of Entropy and Information [5] where he proposes his entropy generalization. He builds his entropy off of a key remark: “the entropy of a combined experiment consisting of the performance of two independent experiments is equal to the sum of the entropies of these two experiments.” Because I am drawing my sample data independently at random, this greatly simplifies entropy calculation. By this additivity of Rényi entropy under independence ((1.20) in [5]), applying it inductively to n i.i.d. draws gives $H_p(X) = n \cdot H_p(x_{\text{single}})$. The same follows for $H_p(X_r) = r \cdot H_p(x_{\text{single} \in R})$. I created a utility function to calculate entropy through this shortcut called `compute_fast_iid_entropy`. It functions much the same as `compute_p_entropy` except it computes the entropy over the probability distribution that the data is drawn from rather than over the data itself. Because all n data points are drawn independently, this can be accurately scaled up to the entire dataset or r -subset by multiplying by n or r within the test cases.

I am keeping this helper function within `test_icc.py` because, while it is an invaluable tool for proving the scheme’s efficacy in practice, the exact entropy calculations of the

dataset instead of the probability distribution it is drawn from are necessary. So, it is not a true replacement for `compute_p_entropy`.

Table 10 uses the `compute_fast_iid_entropy` scaling shortcut to demonstrate the $m \ll n$ property at scale. For high-entropy data, $H_p(X) \approx n$ cancels the leading n term in Theorem 1, keeping m small. For low-entropy data (skew = 0.95), $H_p(X)$ is much smaller than n , so the bound correctly forces $m \approx n$. This corroborates what was shown in Section 2.4.

Table 10: Key size m vs. dataset size n for i.i.d. data ($q = 31$, $p = 2$, $\varepsilon = 10^{-6}$).

n	r	Skew	$H_p(X)$	m	$m \ll n?$
10,000	50	0.10	9,999.7	57	Yes
100,000	100	0.10	99,996.5	110	Yes
500,000	500	0.05	496,400.4	4,102	Yes
100,000	100	0.95	34,971.1	65,070	No

5 Further Work

5.1 Straggler Resilience via Information Super-Sets

The obvious next direction to take this implementation is to fully realize the ICC scheme and implement straggler resilience via information super-sets. As previously mentioned, a multiset $T \in [n]$ is an S -information super-set if any $(|T| - S)$ -subset of T contains an information set. This is given in Definition 4 in [3]. So, even if S workers fail to respond, the remaining workers who do respond have evaluations on an information set so their responses are sufficient for the client to decode the answer.

I have begun a first pass at information super-set generation through creating a `get_information_superset` function in `utils.py`. This function takes a `Galois field (GF)`, field size q , number of variables m , maximum total degree d , and number of stragglers S as parameters. It returns a list which is the superset containing entries such that any subset of size λ contain an information set. Inefficient describes this construction too kindly; it brute-force builds the superset from the ground up by ensuring that no invariants are violated. It begins with the information set I have been relying upon which is created by calling `get_information_set`. It then enumerates all possible points with `itertools.product`. For each of these points, it checks if it is already in the superset and

if not, tentatively adds it. It then ensures that every possible subset of size λ maintains full rank given this point was added. If this passes, the point is added.

While this is a highly inefficient construction, it serves as a good jumping off point to expanding this scheme to tolerate straggling workers.

5.2 Case Study

As another part of my thesis I want to apply this ICC scheme onto a machine learning dataset so this is one more future path for the project. I will take a dataset I want to execute a learning algorithm on and follow the encoding, computation, and decoding phases of this scheme instead of directly performing the learning algorithm on the data. I will use a learning algorithm that can be accurately computed as a polynomial to complete the entire learning stage as if the data has been encoded and offloaded to a cloud environment.

The most interesting part of this case study will be analyzing the information leakage bounds. I will explore how reliable they are, how consistent they are, and whether they are meaningful enough to provide justification for this scheme being a competitive way to train ML models securely without the computation power to do so on their own.

References

- [1] S. Tarnopolsky, Z. Deng, V. Ramkumar, N. Raviv, and A. Cohen, “Individual confidential computing of polynomials over non-uniform information,” 2025.
- [2] N. Raviv and Z. Goldfeld, “Perfect subset privacy for data sharing and learning,” 2022.
- [3] Z. Deng, V. Ramkumar, and N. Raviv, “Perfect subset privacy in polynomial computation,” 2024.
- [4] M. Hostetter, “Galois: A performant NumPy extension for Galois fields,” 11 2020.
- [5] A. Rényi, “On measures of entropy and information,” 1961.